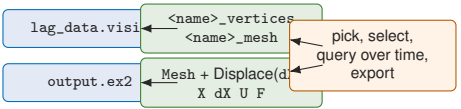
VISIT: LAGRANGIAN AND FINITE-ELEMENT DATA

The Silo files of a point-method run and the ExodusII files of an IBFE run: what is in them, how to draw the structure where it is, colour it by its own fields, pick, select, query it over time, and get it out



pictures: VisIt 3.5.0 on IBFE/01, IBFE/08, IBFE/10, upstream ex7, IB/13 and IB/08;
windows: the 2013, 2014 and 2018 decks

Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part V • Chapter 28

Claude (Anthropic) was used to organize, combine, and format more than fourteen years (2012–2026) of the instructor's tutorial notes from courses and workshops at several universities.

LEARNING GOALS

By the end of this chapter you should be able to:

1. say what a point-method run writes for VisIt about its structure (the Silo files behind `lag_data.visit`: positions only, as `<name>_vertices` and `<name>_mesh`) and what an IBFE run writes (one ExodusII file per part with the reference mesh and the nodal fields `X`, `dX`, `U`, `F`);
2. draw a finite-element mesh where it is now (the Displace operator with the displacement `dX`), on top of the fluid, for one part, several parts, and in 3D;
3. colour the mesh by its own fields and by expressions of them, draw its velocity as vectors, and judge its element quality with VisIt's `mesh_quality` expressions;
4. read numbers off the structure: Pick a node, query the maximum, minimum or average of a field over the mesh or over a selected part of it, and plot that quantity against time;
5. build the displacement of an IB point set from its positions when there is no `dX`, with a cross-time expression;
6. export a mesh with its fields to VTK and a time query to a `.curve` file, and know which chapter reads them next (29, 32, 33).

> The plot pictures are new (VisIt 3.5.0 headless on this series' runs; `wsl/final/visit_ch28.sh`); the window screenshots come from the 2013, 2014 and 2018 decks (VisIt 2.x on a Mac) and are credited on each slide; the 3.x names are in the text. Everything stated about the files was checked against the IBAMR 0.19.0 source and VisIt's own metadata of the files.

WHERE THIS CHAPTER COMES FROM

- "IBAMR Finite Element Visualization in VisIt" (Alex Hoover, Miller Lab, 23 May 2013; a Word document, and a 7-slide deck titled "…with VisIt"): the recipe every later version repeats: open `dumps.visit` and `output.ex2`, untick **Apply operators to all plots**, Mesh plot, **Operators** → **Transforms** → **Displace**, create the vector expression `dX`, save it as XML, Project for the old 2D-in-3D output, and the examples list. Re-issued on the Drive as "0-IBFE Tutorial: Visualizing FE boundaries with VisIt" (the document, 2017; the slides, edited to 2024), as deck 3 of the 2018–19 Visualization and Data Analysis series, as "Lecture 3" of the 2020 visualization series, and as slides 18–26 of the 2020/2024 IBFE Lecture 1 (chapter 17 describes that deck).
- "5-IBFE Tutorial: Visualizing Lagrangian Data" (Hoover, 2014; 9 slides, VisIt 2.7.2): the fields in the Exodus file, query over time, selection, expressions, on a 3D jellyfish-bell ring; its nine screenshots are reused.
- "3B – Analyzing your IBFE simulation" (MATH 769, 2018; 16 slides, VisIt 2.12.3 on KillDevil): recording the GUI to Python, exporting `.ex2` as VTK for a cluster whose VisIt could not read Exodus, the Box average over time, the `.curve` file and its MATLAB import. The recording part is chapter 32; the Eulerian box average chapter 29; the Lagrangian export and the curve file are here.
- MATH 769 "IBAMR with FE" (April 2013), slides 9–10: Boyce Griffith's description of the file from the IBAMR mailing list, quoted on slide 17.
- New here: the Silo writer read from the source (`LSiloDataWriter.cpp`, `IBRedundantInitializer.cpp`): the file layout, the names, what is and is not written; VisIt 3.5's metadata of each file (the vector expressions it now builds itself; `mesh_quality`); the Pick output; the domain Subset; a cross-time displacement for IB points; the error texts; the 3D and multi-part cases on this series' runs; the exported files shown as they come out.

TWO KINDS OF LAGRANGIAN OUTPUT

method	files for VisIt	what they hold
IB (points, springs, beams; Parts II–III)	`viz_IB2d/lag_data.visit` and the `lag_data.cycle_NNNNNN/` folders (Silo)	the positions of every point at every visualization dump, as a point mesh and as the edges of the springs; no velocity, no force (slide 8)
IBFE (finite elements; Part IV)	`viz_IB2d/output.ex2`, one ExodusII file per part (`beam_output.ex2`, `circle_output.ex2`, ...)	the reference mesh and, at every dump, the nodal fields: position `X`, displacement `dX`, velocity `U`, force density `F` (slide 15)
ConstraintIB (chapter 14)	the same Silo files for the points	the kinematics, forces and energies are text files in `PrintOutput/`
IB2d (Part II)	`viz_IB2d/lagsPts.NNNN.vtk` (and `lagsPtsConnect` when asked)	the points as VTK; chapter 26 drew them

- The forces and velocities of a point-method run are *not* in its VisIt files: they are the `hier_data` text files of chapter 11 (`X.*`, `F.*`), or the sums the program prints. VisIt shows where the points are.
- An IBFE run has the opposite arrangement: everything about the structure is in the Exodus file, and there is no `lag_data.visit`.
- Both kinds sit next to the fluid's `dumps.visit` (chapter 26) and are opened as a second source in the same window.

THE SILO FILES ON DISK

```
viz_IB2d/                          # IB/13 human, 4 ranks
  dumps.visit                      # fluid (ch. 26)
  lag_data.visit                   # 102 lines: one folder per frame
  lag_data.cycle_000000/
  lag_data.cycle_000341/
    lag_data.cycle_000341.summary.silo   # 2.8 kB
    lag_data.proc_0000.silo              # 30 kB
    lag_data.proc_0001.silo              # 1 kB
    lag_data.proc_0002.silo
    lag_data.proc_0003.silo
  lag_data.cycle_000682/ ...
```

```
Main {                          // what controls them
   viz_writer = "VisIt","Silo"   // "Silo" switches the writer on
   viz_dump_interval = 341
   viz_dump_dirname = "viz_IB2d"
}
```

- The names are fixed in `LSiloDataWriter.cpp`: `lag_data.visit`, the folder prefix `lag_data.cycle_`, the summary file `<folder>.summary.silo`, one `lag_data.proc_PPPP.silo` per MPI rank. The folder number is the time step, not the frame number.
- Each rank writes its own file; the summary file (written by rank 0) is a Silo *multimesh* that lists them, and that is what `lag_data.visit` points at. Open the `.visit` file; never a `proc` file.
- Why rank 0's file is big and the others are small: the initializer registers every structure on rank 0 (slide 8), and the writer gathers that structure's points to the rank that registered it (a `VecScatter` from the owning ranks) before writing. The other ranks' files are empty Silo files (a root directory and nothing else; 1 kB each here). The points still live on the ranks that own their patches during the run.

WHO WRITES THE SILO FILES, AND WHAT SWITCHES THEM ON

- Without `"Silo"` in `viz_writer` no Lagrangian file is written; an IB/01-style program then also segfaults at its first dump (chapter 16's silent-failures table), because `getLSiloDataWriter()` returns null and the program dereferences it.
- The writer is created and registered in the program: `silo_data_writer = app_initializer->getLSiloDataWriter()` and `ib_method_ops->registerLSiloDataWriter(silo_data_writer)` (every example's `main.cpp`; chapter 11 lists the outputs). The `LDataManager` then hands it the coordinates of every level (`registerCoordsData`) and the Lagrangian ordering (`registerLagrangianAO`); the initializer tells it which index ranges are structures and which edges are springs (next slide).
- The writer's own API could add fields to the points, `registerVariableData(name, LData, level)`, but `IBMethod` never calls it: a custom program can (chapter 11's output hooks are the place), and `GeneralizedIBMethod`, `IMPMethod`, `PenaltyIBMethod` (`Y`, `M`) and `CIBMethod` (`lambda`) do (slide 9).

WHEN THE SILO FILES ARE WRITTEN, AND WHAT A RESTART KEEPS

- `silo_data_writer->writePlotData(iteration_num, loop_time)` is called by the program's main loop (`main.cpp`), in the same `if` block as the fluid writer's `visit_data_writer->writePlotData(...)`, at every `viz_dump_interval` and at the last step; no integrator calls it. That is why the two time series have the same frames.
- The restart files hold the writer's registrations (structure names, index ranges, edge maps: `putToDatabase`), which is why the initializer does not re-register them after a restart. The folder numbers continue because they are the iteration numbers the program passes; `lag_data.visit` itself is truncated at the first post-restart dump, so chapter 3's rebuild recipe applies to it as to `dumps.visit`.
- A frame is therefore the pair (SAMRAI dump, Silo dump) at one iteration; `viz_dump_interval` in `Main` is the only cadence, and `data_dump_interval` (the `hier_data` text files of chapter 11) is independent of it.

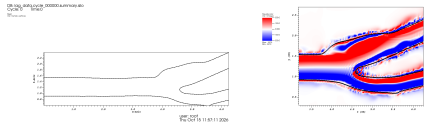
WHAT IS INSIDE: NAMES AND CONTENTS

Silo object	what it is (from `IBRedundantInitializer::initializeLSiloDataWriter`)
`<structure>_vertices`	a *marker cloud*: the points of one structure file (`<structure>.vertex`), a 0-dimensional point mesh; registered for every structure
`<structure>_mesh`	an *unstructured mesh* whose edges are the springs of `<structure>.spring`; registered only when the structure has springs; the Mesh plot of it with a line width is the connected boundary of chapter 26
`<structure>_xmesh`	the same for the cross-links of a `.xspring` file
`<structure>_rod_mesh`	the rods of a `.rod` file; named `_mesh` instead when no structure on that level registered a spring mesh (the flag is per level, not per structure)
beams, targets, anchors, masses	not drawn: no edge map is registered for them; a beam-only structure shows as points
`level_L_cloud_K`, `level_L_mesh_K`	the directories inside each `proc` file (one per level, cloud or mesh); the summary's multimesh gathers them under the structure's name
`domains`, `blocks`	the Subset categories VisIt makes from the multimesh: one domain per registered mesh (so one, `domain1`, for a structure registered on rank 0), not one per rank

WHAT IS WRITTEN FOR THE POINTS: VARIABLES, RANKS, ORDER

- Variables: `LDataManager` registers the coordinates (`registerCoordsData`) and nothing else for `IBMethod`; VisIt's metadata of `lag_data.visit` lists no scalars and no vectors. `GeneralizedIBMethod` adds the directors `D1`, `D2`, `D3` and the angular velocity `W`; `IMPMethod` adds `F0`–`F2` and `tau0`–`tau2`; `PenaltyIBMethod` (chapter 13) adds `Y` and `M`, `CIBMethod` `lambda`. So "colour the points by their force" is not possible from these files for the plain IB method: chapter 11's `hier_data` files are the route, or a cross-time expression for the displacement (slide 12).
- Who registers: the initializer does it on MPI rank 0 only ("this will fail if the structure is too large to be stored in the memory available to a single MPI process", the source's own warning); the names are the structure's base file name plus the postfix.
- Order: the writer keeps the Lagrangian numbering through the application ordering it is handed (`registerLagrangianAO`), so node k of a frame is the point of line $k + 2$ of the `.vertex` file (line 1 is the count) at every frame (checked on slide 11).

VISIT'S VIEW OF `LAG_DATA.VISIT`



IB/13 human (chapter 26): the Mesh plot of `human_vertices` (points) and of `human_mesh` (the spring edges, line width 3).

- Metadata (**File** → **File information**, or `GetMetaData` in the CLI): meshes `human_mesh` and `human_vertices`, 102 states with their times, one block each (block title `domains`, group title `blocks`), no scalars, no vectors, and the `mesh_quality` expressions VisIt offers for any mesh.
- **Add** → **Mesh** draws either (chapter 26). **Add** → **Subset** → **domains** exists but colours everything alike: the structure was registered on one rank and is one domain (`domain1`); a run with two structure files has two meshes, not two domains.
- What is *not* there: no velocity, no force, no displacement. The `time_derivative` expressions VisIt lists for a mesh (`time(human_vertices)`, `conn_cmfe(...)`) are the building blocks of slide 12.
- A structure whose file has beams but no springs shows as points only; add a `.spring` file with zero stiffness if you want the edges drawn (chapter 6's format; stiffness 0 changes nothing in the run).

PICKING A POINT: NODE NUMBER = LAGRANGIAN INDEX

```
# CLI: PickByNode(domain=0, element=0) on human_vertices, frame 0
   (positionally the node comes FIRST: PickByNode(node, domain))
A:  lag_data.cycle_000000.summary.silo  timestep 0
human_vertices
Point: <0.0078125, 0.920573>
Node:  0
Incident Zones:  0
```

```
# human.vertex, the structure file (chapter 6)
1045
7.8125000000000000e-03 9.2057340405399213e-01   # line 2 = node 0
2.3447577247806096e-02 9.2050862810363454e-01   # line 3 = node 1
```

- In the GUI: **Controls** → **Pick**, **Node** mode, click a point; the Pick window prints its coordinates and node number, and a letter appears at the point. On a point mesh every point is also a one-node zone, so a Zone pick returns the same point (zone k = node k).
- The node number is the Lagrangian index within the structure (0-based; the `.vertex` file's line $k+2$), the same number `getNodeDataItem` (chapter 9a) and the `X.*` files of chapter 11 use. Two structures in one run (`human` and a second file) have separate meshes and separate numberings.
- **Do time curve** in the Pick window turns the next pick into the history of that node: one curve per coordinate in window 2 (**Multiple Y axes** or one). This is the quickest "where did point 512 go" tool; the scripted version (`PickByNode` in a loop over states) is chapter 32.
- `NumNodes` on the point mesh gives 1045, the file's first line; the 2D Silo picture needs nothing else: the Pick confirms that node 0 is the file's first point, $(0.0078125, 0.920573)$.

A DISPLACEMENT FOR IB POINTS: THE CROSS-TIME EXPRESSION

```
# Controls > Expressions (three of them, in this order)
pos       vector  coords(human_vertices)
pos0      vector  conn_cmfe(<[0]i:pos>, human_vertices)
disp_mag  scalar  magnitude(pos - pos0)
```



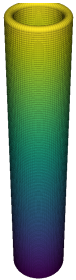
- `coords(mesh)` is the position of each node as a vector; `conn_cmfe(<[0]i:var>, mesh)` fetches `var` from state 0 (`[0]i` = absolute index 0; `[-1]id` would be the previous state) and maps it node by node, which is right because the writer keeps the numbering (slide 8). The difference is the displacement since the first frame; its magnitude is a scalar to colour the points by.
- Picture: IB/13 human at frame 56; the displacement since $t = 0$ runs from 3×10^{-5} to 0.029 cm (its mean is well under a cell: the walls are target points), largest at the open ends where the outlet bands pull. The target points of this example barely move, which is the point of the picture: the expression measures how far the walls drift from their targets (chapter 9a's question).

THE CROSS-TIME EXPRESSION: VARIANTS

```
pos_prev  vector  conn_cmfe(<[-1]id:pos>, human_vertices)   # the previous frame ([-1]id = relative index)
step_mag  scalar  magnitude(pos - pos_prev)                 # displacement per frame
speed_est scalar  magnitude(pos - pos_prev) / 0.01705       # divided by the frame interval: a velocity estimate
pos_t1    vector  conn_cmfe(<[56]i:pos>, human_vertices)    # any absolute frame, here the systolic peak
```

- The same three lines with `[-1]id` give the displacement per frame, and divided by the frame interval, a velocity estimate at the dump cadence (coarse: 0.017 s here, against the run's 5×10^{-5} s step); the `X.*` files of chapter 11 give the exact velocity at the data-dump cadence instead.
- `conn_cmfe` maps by connectivity (node k to node k) and needs the same number of points in both frames, which a point-method run guarantees; `pos_cmfe(<[0]i:var>, mesh, fill)` maps by position and is for meshes that change, or for sampling the fluid onto the structure (chapter 29).
- Expressions live in the session, not in the data: save the session (chapter 26) and **Restore session with sources** on the next run; or keep them in a script (`DefineVectorExpression`, chapter 32).
- The GUI's **Controls** → **Expressions** window has a **Time iteration** tab for the same thing ("this variable at another time"); the typed form above is the one that scripts and sessions store.

A 3D POINT CLOUD: IB/08 GLYCOCALYX



`glyco_512_vertices` as spheres coloured by height (`coords(glyco_512_vertices)[2]`) at $t = 0.2$: a hollow cylinder of points standing on the floor of the box.

- A 3D run writes the same files into `viz_IB3d/`; the cloud is a 3D point mesh. This structure has no springs (target points only), so there is no `_mesh`: the Mesh plot of `glyco_512_mesh` that chapter 12's table named does not exist ("An invalid variable (glyco_512_mesh) was specified").
- Points in 3D are invisible at the default size: in the Pseudocolor or Mesh attributes set **Point type** to **Sphere** and a **Point size** in data units (or pixels); the Mesh plot has the same controls.
- Camera, slices and the 3D view are chapter 27; everything else on these slides (pick, subset, expressions, queries) works unchanged in 3D.
- The glycocalyx of IB/08 is target points without springs, so the file has `glyco_512_vertices` and no `_mesh`: $13\,600$ points on a cylinder 0.026 across and 0.156 tall at the bottom of a box 1 high. VisIt's `ResetView` frames the plots that exist, so draw the points first and add the fluid afterwards when the structure is a small part of the box.

THE EXODUSII FILE: WHAT LIBMESH WRITES FOR IBFE

system (`FEMechanicsBase.cpp`)	variables	meaning
`IB coordinates system`	`X_0`, `X_1` (, `X_2`)	the current position of each node
`IB coordinate mapping system`	`dX_0`, `dX_1` (, `dX_2`)	the displacement from the reference configuration, $\mathbf{X} - \mathbf{s}$: what the Displace operator wants
`IB velocity system`	`U_0`, `U_1` (, `U_2`)	the node's velocity, interpolated from the fluid ("the velocity exerted on the mesh" of the 2014 deck)
`IB force system`	`F_0`, `F_1` (, `F_2`)	the Lagrangian force density (stress divergence, tethers, body forces)
`IB pressure system`	`P`, or `Phi`	`P` with a static or dynamic pressure part (`FEMechanicsBase`); `Phi` with `IBFEMethod`'s stress-normalization part (upstream ex0 uses it), written under the same system name; neither in this series' runs
`IB source system`	`Q`	a Lagrangian body source (`registerLagBodySourceFunction`); not in this series' runs

- `exodus_io->write_timestep(filename, *equation_systems, n, time)` writes *every* system's variables as nodal fields, one state per visualization dump numbered from 1 (chapter 17 showed the call and `exodus_io->append(from_restart)`, which continues the file after a restart instead of starting it over).

THE EXODUSII FILE (2): THE MESH, THE PARTS, AND WHAT SWITCHES IT ON

- The mesh itself is written once, in its reference configuration, with its element blocks (the mesher's physical groups or Exodus blocks of chapter 18: one block `0` for the plate), node sets and side sets (upstream ex7's walls carry node sets).
- One file per part (chapter 21): `getExodusIIFilename("beam_")` gives `viz_IB2d/beam_output.ex2`. The file is written by rank 0 from the gathered mesh, so it is one file whatever the rank count.
- `viz_writer = "VisIt","ExodusII"` is what the examples write, but in 0.19.0 the second word is inert: `AppInitializer` starts with `d_exodus_filename = "output.ex2"` and `getExodusIIFilename()` returns it whether or not `"ExodusII"` is listed (the entry only lets an `exodus_filename` key rename the file). The plate file is written whenever visualization dumps are on (`viz_dump_interval > 0`) and the program calls `write_timestep`; a test run with `viz_writer = "VisIt"` alone wrote `output.ex2`. Chapter 17's "both words or no plate file" is corrected with this chapter.

BOYCE GRIFFITH ON THE FILE (2013)

> *"At present, with the IBFE examples, there is a file created in the viz directory named* `output.ex2`*. This is an ExodusII file. The file stores the reference configuration of the mesh (which need not be the initial configuration) as well as field data defined on the mesh, including the Lagrangian velocity field and force density. I do not know how to setup these files so that VisIt automatically recognizes that the individual components of a field belong to a vector field variable. If you want to visualize vector fields on the mesh, you have to set up a vector expression. E.g., you may wish to set up a vector expression* `U` *as* `{U_0, U_1, U_2}` *(in 3D) or* `{U_0, U_1, 0}` *(in 2D). If you want to visualize the current configuration of the mesh, you need to set up a vector expression* `dX = {dX_0, dX_1, dX_2}` *for the displacement between the reference and current configurations. You can then apply the 'displace' operator to the mesh. Currently, 2D output is really stored in 3D, but with a trivial z component. If you want to simultaneously visualize 2D Eulerian and 2D FE data, you will need to apply the 'project' operator to the FE data. Note that you will need to unselect 'Apply operators to all plots' to prevent the displace and project operators from being applied to the Eulerian data."*

- Quoted without attribution in the MATH 769 deck of April 2013; by its content and its first-person voice it is Boyce Griffith's, presumably from the IBAMR users' list (the post itself was not found). Three things have changed since, and the next slide shows them in VisIt's own metadata: the vector expressions are built by VisIt now, 2D output is two-dimensional, and the Project step is gone. The rest is still exactly right.

MATH769_IBFE.pptx (2013), slides 9–10; text attributed to B. Griffith.

VISIT'S VIEW OF OUTPUT.EX2 (IBFE/01, 3.5.0)

```
states 41, times 0.0, 0.1, ... 4.0
mesh    Mesh   spatial dim 2, topological dim 2, 1 block
scalars ElementBlock (zonal)
        F_0 F_1  U_0 U_1  X_0 X_1  dX_0 dX_1   (nodal)
        coordx coordy  node_num_map  elem_num_map
vectors (none in the file)
expressions, made by VisIt on opening:
        F  = {F_0, F_1}     U  = {U_0, U_1}
        X  = {X_0, X_1}     dX = {dX_0, dX_1}
        mesh_quality/area, aspect, condition, degree, jacobian,
        max_edge_length, maximum_angle, min_edge_length,
        min_corner_area, min_sin_corner, minimum_angle, oddy,
        relative_size, scaled_jacobian, shape, shear, skew, ...
NumNodes 101, NumZones 130
```

- The Exodus reader of VisIt 3.5 groups variables named `name_0`, `name_1` into a vector expression `name` by itself in the 2D files (not in the 3D file: slide 40): `dX` is already under **Vectors** in the Displace window, as the 2024 edit of the deck noticed ("for newer IBAMR versions, there should be a dX option under Vectors"). Creating it by hand (slide 19) still works and does no harm.
- `coordx`, `coordy` are the reference coordinates as scalars (useful in expressions); `node_num_map`, `elem_num_map` are libMesh's ids, which match the mesh file.
- The mesh is two-dimensional (`coordz` absent), so a 2D fluid and a 2D mesh share a window with no Project.
- `ElementBlock` is a zonal scalar and a Subset category (slide 38); the 3D file (slide 40) has the same layout with `_2` components and 3D quality measures.

THE RECIPE: THE MESH WHERE IT IS NOW

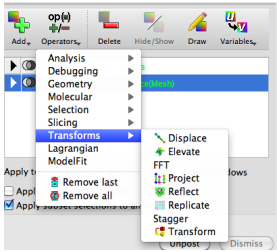
1. **Open** `viz_IB2d/dumps.visit`; **Add** → **Pseudocolor** → **Omega**; **Draw**.
2. **Open** `viz_IB2d/output.ex2` (say yes to the correlation); with it as the **Active source**, **Add** → **Mesh** → **Mesh**. Do not Draw yet.
3. Untick **Apply operators to all plots** (bottom left of the plot list).
4. Select the Mesh plot; **Operators** → **Transforms** → **Displace**; expand the plot, double-click **Displace**: **Displacement variable** → **Vectors** → **dX**, multiplier 1; **Apply**.
5. If `dX` is not listed (older VisIt, or any 3D file): **Create new expression**, name `dX`, type *Vector mesh variable*, definition `{dX_0, dX_1}` (3D: `{dX_0, dX_1, dX_2}`); **Apply** in the Expressions window; then pick it.
6. **Draw**. Step through time: the mesh follows the fluid.



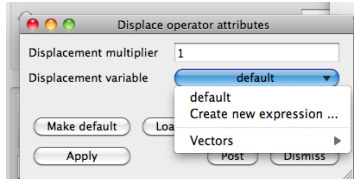
IBFE/01 FULL at $t = 1$: before and after the Displace operator (chapter 17's pictures).

- Hoover's 2013 steps, in 3.5's words. The old "Project" step (**Operators** → **Transforms** → **Project**, xy-plane by default; "you can change this if you want if you double click Project in the collapsed menu from the Mesh Object") was for output that stored 2D meshes with three coordinates; it is not needed now and does nothing harmful if applied.
- Save the session (**File** → **Save session**) once the plots are right; restoring it with sources on the next run replaces steps 2–5.

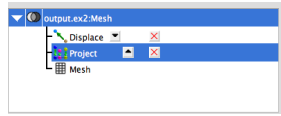
THE WINDOWS (2013)



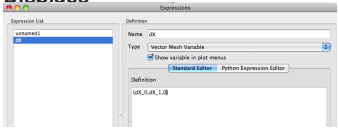
Operators → Transforms → Displace



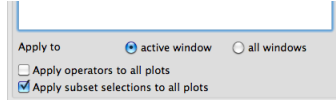
the Displace attributes: multiplier, variable (**default**, **Create new expression**, **Vectors** →)



the plot list with Displace and the old Project on the Mesh plot



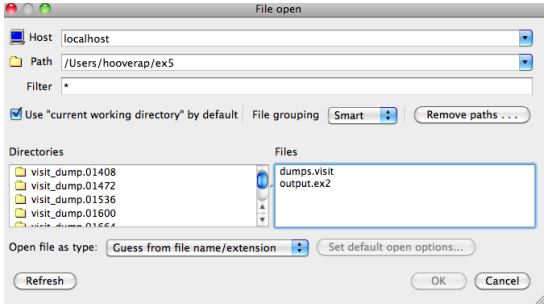
the Expressions window: `dX`, Vector Mesh Variable, `{dX_0, dX_1, 0}` (the 2013 3-component form)



the two checkboxes under the plot list: untick the first

A. Hoover, IBAMR Finite Element Visualization with VisIt (2013), VisIt 2.x on a Mac; the same windows in 3.5 with the Vector submenu filled in.

THE WINDOWS (2013), CONTINUED: THE FOLDER



the File open window of 2013: `dumps.visit` and `output.ex2` beside the `visit_dump.NNNNN` folders

- The window documents the folder layout of an IBFE run as it was in 2013 and still is: the fluid frames as `visit_dump.NNNNN` folders listed by `dumps.visit`, and the one Exodus file beside them; the path (`/Users/hooverap/ex5`) records that the walk-through was made on upstream ex5, the cylinder.
- Open the two files one after the other (**Open** twice); the second open asks for the database correlation (chapter 26); each becomes an entry of the **Active source** list.
- “Open file as type: Guess from file name/extension” is right for both; the `.ex2` extension selects the Exodus reader, `.visit` the SAMRAI one.

A. Hoover (2013), VisIt 2.x on a Mac.

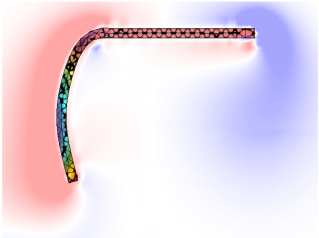
“APPLY OPERATORS TO ALL PLOTS”: THE ERROR YOU GET

```
VisIt: Error - Pseudocolor:  ()
viewer: An invalid variable (dX) was specified.
```

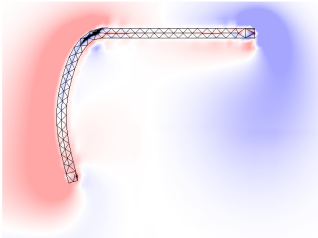
What VisIt 3.5 says when the Displace operator with variable `dX` lands on the fluid’s Pseudocolor plot as well: `dX` exists on the Exodus mesh, not on `amr_mesh`, so the fluid plot cannot be drawn and nothing appears. (Reproduced on purpose; `wsl/final/visit_ch28.sh`.)

- The checkbox is a convenience for windows whose plots share a database. With two sources it is a trap, which is why every version of the recipe starts with “untick”.
- Fix: select the fluid plot, **Operators** → **Remove last** (or **Remove all**), untick the box, re-apply Displace to the mesh plots.
- The reverse mistake, Displace on the Mesh but a Pseudocolor of `F_1` on the same file without it: the colours are drawn on the reference mesh while the Mesh plot has moved. Every plot that should move gets its own Displace (slide 23).
- The multiplier is a scale on `dX`: 1 shows the true position; 10 exaggerates a small deformation (useful on stiff bodies, but say so on the figure); 0 is the reference mesh again.

COLOURING THE MESH BY ITS OWN FIELDS



IBFE/01 HALF at $t = 1$: `magnitude(dX)` (0.25 to 0.5) on the displaced plate over the vorticity; the tethered right half has moved 0.5 up, the free left half hangs from it (the README's $39°$ tip-to-tip tilt). The query on this Pseudocolor plot: $|\mathbf{dX}|$ from 0.271 (the hanging half's middle) to 0.501, mean 0.434.

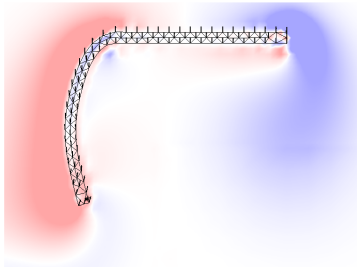


the same frame, `F_1` (the vertical force density, ± 800) on the plate: from -758 at the tethered tip (node 0, pulled back against the tether's overshoot) to $+1122$ at the next node, and near zero on the free half, which carries no tether.

COLOURING THE MESH: HOW

- With the Exodus file as the active source, **Add** → **Pseudocolor** → **dX_mag** (an expression, `magnitude(dX)`; or any of `dX_0`, `U_1`, `F_0`, `...` directly), then **Operators** → **Transforms** → **Displace** on *this* plot too; the Mesh plot on top with **Opaque mode: Off** draws the edges over the colours. Fix the limits (**Minimum**, **Maximum**) as in chapter 26, or the colours flicker from frame to frame.
- `F` is the force density the structure exerts on the fluid at the node (per unit reference area in 2D); its sign and size tell where the tether works hardest. `U` is the velocity the node actually has, interpolated from the fluid.
- Nodal variables on a triangle mesh are interpolated linearly across each element: there is no centering choice to make, unlike chapter 26's fluid fields; a thin plate needs a zoom (this window is 1×0.75 of a 2×2 box) or thicker lines before the colours can be seen at all.
- The fluid's vorticity behind it is a Pseudocolor on `dumps.visit` with its opacity lowered (**Opacity** 35%), so the structure's colours stay readable.

VECTORS ON THE MESH



- **Add** → **Vector** → **U** on `output.ex2`, every node (**Stride** 1), one colour, with the Displace operator on the Vector plot too: the plate's velocity at $t = 1$ (HALF: the tethered half moves at the prescribed 0.5, the free half swings at up to 0.61; the **MinMax** query on the Vector plot gives 0.4997 to 0.608).
- The Vector plot draws one glyph per node of the mesh (101 here), so **Stride** 1 is right on a structure, where chapter 26 thinned the fluid's vectors to a few thousand; **Scale** sets the arrow length in data units (0.12 here).
- The Vector plot of `F` shows the force density the same way; on a tethered plate it is large only where the tether pulls.
- In 3D the expression must be built first (`{U_0, U_1, U_2}`, slide 40).

THE MESH'S OWN QUALITY



`mesh_quality/minimum_angle` of the plate's 130 triangles at the reference state, 27° to 54° (the `aspect` measure returns 0 on triangles in 3.5; `condition` runs from 1.007 to 1.42): the expressions VisIt attaches to every mesh.

- `mesh_quality/*` are computed from the geometry at the current state: at the reference state they judge the mesher's work (chapter 18 for the element size, chapter 22's sliver slide for the shape: angles well away from 0 and 180°, no slivers); at a later state, `jacobian` or `scaled_jacobian` going to zero shows an element that has been crushed, and `area` (`volume` in 3D) against its reference value is the local J of chapter 19.
- A Threshold on one of them (slide 33) isolates the bad elements; the 3D plate of chapter 22 with its sliver tetrahedra is on slide 40.

THE MESH_QUALITY EXPRESSIONS, 2D AND 3D

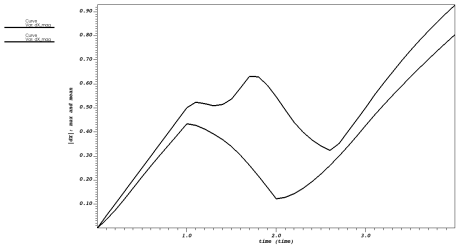
- The 2D list (from the file's metadata): `area`, `aspect`, `condition`, `degree`, `jacobian`, `max_edge_length`, `maximum_angle`, `min_edge_length`, `min_corner_area`, `min_sin_corner`, `min_sin_corner_cw`, `minimum_angle`, `neighbor`, `oddy`, `relative_size`, `scaled_jacobian`, `shape`, `shape_and_size`, `shear`, `skew`, `stretch`, `taper`, `warpage`. The 3D list drops `area`, the angle measures, `min_corner_area`, `min_sin_corner(_cw)` and `warpage`, and adds `aspect_gamma`, `diagonal_ratio`, `min_diagonal`, `max_diagonal`, `dimension`, `max_side_volume`, `min_side_volume`, `node_degree`, `volume`, `face_planarity`, `relative_face_planarity`: in 3D judge tetrahedra by `aspect_gamma`, `condition` or `shape`.
- Each is a zonal scalar (one value per element), so a Pseudocolor of it colours elements, and a Threshold on it keeps elements; the Query window's **Min**/**Max** give the worst element and its position.
- They are VisIt's (the Verdict library), not IBAMR's: the same names appear on any mesh VisIt opens, including the fluid's `amr_mesh`, where they say nothing useful (every cell is a square).

PICK: NUMBERS AT ONE NODE

```
B:  output.ex2  timestep 10        # IBFE/01 FULL, t = 1
Mesh
Point: <0.5, 0>                    # reference position
Node:  0
Incident Zones:  0  1  3
X_0:  <nodal> = 0.499631
X_1:  <nodal> = 0.500316
dX_0:  <nodal> = -0.000369177
dX_1:  <nodal> = 0.500316
U_0:  <nodal> = 0.00198706
U_1:  <nodal> = 0.510143
F_0:  <nodal> = 530.064
F_1:  <nodal> = -652.487
```

- **Controls** → **Pick**, **Node** or **Zone** mode, choose the variables to report, click on the plot (or type a node number in the window's **Pick by node** field). The output above is the CLI's `PickByNode(0, 0, (...))`: node 0 is the plate's right tip, at the reference point $(0.5, 0)$; at $t = 1$ it sits at $(0.4996, 0.5003)$, moving up at 0.51, pushed by a tether force of $(530, -652)$.
- The reported **Point** is always the *reference* (file) position, with or without the Displace operator on the plot; the mouse pick is made in the displayed geometry, so click where you look, and read the current position from `X_0`, `X_1`.
- **Do time curve**: the next pick produces the chosen variables of that node against time in window 2, a Curve plot that **File** → **Save window** exports as text (slide 44). This is how the IBFE/01 README's tip positions were read.
- Node numbers are libMesh's (0-based here; `node_num_map` holds the file's own ids). To find a node by position, pick it once with the mouse; the number is then yours for scripts.

QUERY OVER TIME: A FIELD'S MAXIMUM, MINIMUM OR MEAN AGAINST TIME



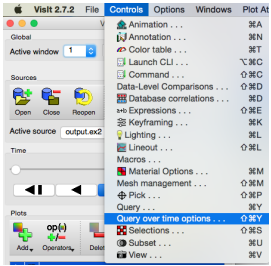
Window 2 after two time queries on `magnitude(dX)` of IBFE/01 HALF: the maximum (the tethered tip's prescribed path) and the average over the plate. The maximum follows the prescribed tether only up to $t = 1$ (0.05 per 0.1 s to 0.50); from $t \approx 1.1$ the swinging free half moves further than the tethered tip, so the maximum stays above 0.3 (0.55 at $t = 2$, 0.32 at $t = 2.6$, 0.50 at $t = 3$ when the tether reaches -0.5) and climbs to 0.93 at $t = 4$ after the release at $t = 3$; the mean lags it. The `.curve` file on slide 44 is this window.

1. **Controls** → **Query over time options**: x-axis **Time** (called **DTime** in the Python attributes; **Cycle** and **Timestep** are the alternatives), start, end and stride if not the whole run.
2. Select the Pseudocolor plot of the variable (a Mesh plot cannot be queried: "Cannot perform a Max query on variable Mesh"); it must be visible ("cannot be hidden", 2014) and needs no Displace operator ("no need to displace at the moment": queries read the fields, pictures need the operator).

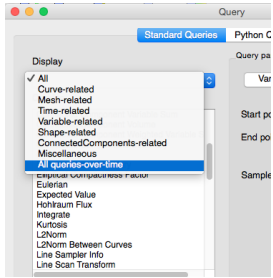
QUERY OVER TIME (2): THE QUERY, AND THE WINDOW IT LANDS IN

3. **Controls** → **Query**: **Display: All queries-over-time**, pick **Max**, **Min** or **Average Value**, tick **Do Time Query**, **Query**.
4. The result opens window 2 as a Curve plot (“this may take a while” over many states; “terrible UI decision” in the 2014 deck: subsequent queries go into the same window, “can probably be fixed using Python”; in 3.5 the Query over time options have the window settings for it). The curve window is bare by default: **Controls** → **Annotation** (the deck’s “Control>Annotations”) sets its title, axes and legend before **File** → **Save window**.

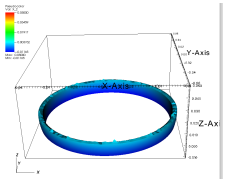
THE 2014 WINDOWS: QUERIES



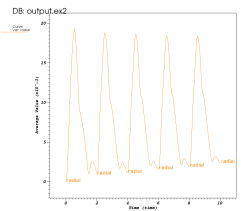
Controls → Query over time options



the Query window, **Display: All queries-over-time**



the bell ring coloured by `X_2`, its z position (slide 7)

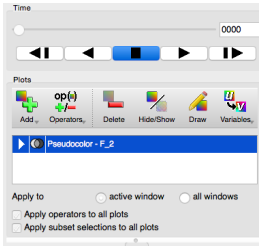


the curve window

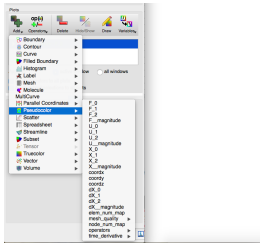
The 2014 deck did this on a jellyfish bell (a 3D ring mesh): the average z position `X_2` of the bell (slide 6) and the average of its "radial" expression over $z < 0.01$ (slide 9), with the windows above.

A. Hoover, 5-IBFE Tutorial: Visualizing Lagrangian Data (2014), VisIt 2.7.2 on a Mac.

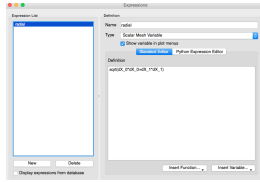
THE 2014 WINDOWS: PLOTS AND EXPRESSIONS



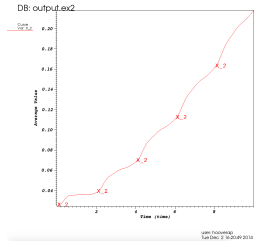
the main window with `F_2`



Add → **Pseudocolor** on an Exodus file



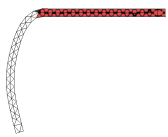
the Expressions window (“radial”)



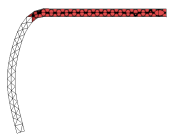
slide 6: the average `X_2` of the bell against time in window 2

A. Hoover (2014), VisIt 2.7.2. The Pseudocolor menu already lists `F_magnitude` and `U_magnitude`: VisIt’s reader built the vectors then too.

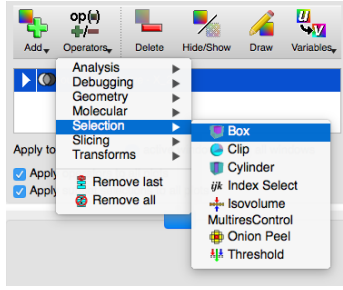
SELECTIONS: QUERYING PART OF THE STRUCTURE



Operators → **Selection** → **Box**, $x \in [0, 1]$: the right half



Threshold on `dX_mag` ≥ 0.49: the nodes that moved the full way



the Selection submenu (2014)

- A selection operator keeps part of the data; every query after it acts on what is left. On the HALF plate at $t = 1$: the Box $x \in [0, 1]$, applied after Displace so it acts on the current positions, keeps 53 of the 101 nodes (66 of 130 triangles) with mean $|\mathbf{dX}| = 0.4999$; the Threshold $|\mathbf{dX}| \geq 0.49$ keeps 56 nodes and 70 triangles, mean 0.4993: the tethered half and the fold.

THE SELECTION OPERATORS

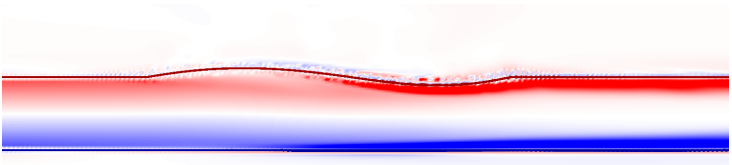
- **Box** takes ranges in x, y and z ("you must specify range in x, y and z", the 2018 deck, of a 3D run; the window asks for z in 2D too, and a range containing 0 is harmless); **Clip** a half-space or a sphere; **Threshold** a range of a variable (zonal: keep an element when all, or part, of its nodes qualify: **Show zone if: Part in range** / **All in range**, `zonePortions` in Python); **Isovolume** the same for a continuous field; **Index Select** and **Onion Peel** by numbering and adjacency; **Cylinder** a tube (3D).
- The 2014 deck selected the bell's lower half "for $z \leq 0$" on one slide and "$z < 0.01$" on another, naming the operator only through its screenshot of the Selection menu with **Box** highlighted; a Box (or Clip) with that z range reproduces it. The "radial component" it then averaged is slide 35.
- The operator applies to the plot it is attached to, so put the selection on the Pseudocolor plot you query, and leave the Mesh plot whole if the picture should still show the entire structure.
- Operator order matters (top of the list is applied first): Displace then Box selects by current position; Box then Displace selects by reference position and moves what is left. The picture on the previous slide used the first order.
- A selection is also the way to a query on one part of a multi-part run when the parts are in one file (one block each): Threshold on `ElementBlock` (slide 38).

EXPRESSIONS ON THE MESH

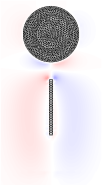
```
# Controls > Expressions on an Exodus file (3.5 defines dX, U, F, X)
dX_mag    scalar  magnitude(dX)                    # in-plane
  displacement
speed     scalar  magnitude(U)
Fmag      scalar  magnitude(F)
Fn        scalar  F_1                              # one component:
  no expression needed
dX_rad    scalar  (dX_0*coordx + dX_1*coordy) / sqrt(coordx*coordx +
   coordy*coordy)
                                                   # radial about the
   origin, from the reference position
dX_frac   scalar  magnitude(dX) / 0.5              # relative to a
  prescribed amplitude
work      scalar  F_0*U_0 + F_1*U_1                # power density at
   the node
jac0      scalar  mesh_quality/jacobian             # element quality
   now
```

- Hoover's 2014 "`sqrt(dX_0*dX_0+dX_1*dX_1)` gives radial component" is the in-plane displacement *magnitude*; for a bell whose axis is z that is the distance moved in the plane, not its radial part. The true radial component projects `dX` on the radial unit vector (the `dX_rad` line, with `coordx`, `coordy` as the reference position; in 3D about the z axis the same two components).
- Expression types: *Scalar mesh variable* or *Vector mesh variable* (the type is the menu it appears in). Vector algebra: `{a, b}`, `v[0]`, `magnitude(v)`, `dot(v, w)`; **Insert Function** and **Insert Variable** write the names for you.
- Expressions on the fluid and on the mesh are separate worlds: an expression that mixes `Omega` and `dX` fails (different meshes). To bring a fluid value onto the structure, VisIt's `pos_cmfe` maps one mesh onto another by position; chapter 29.

SEVERAL PARTS: ONE FILE EACH



Upstream IBFE ex7 (chapter 23) at $t = 2$, the collapsible segment: `loweroutput.ex2` (blue) and `upperoutput.ex2` (red), each a Mesh plot with its own Displace, over the vorticity (± 50).



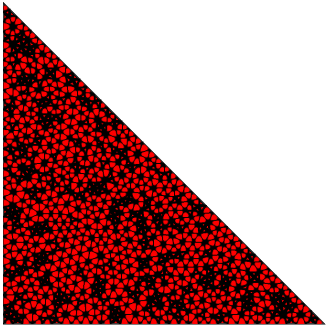
IBFE/08 (chapter 21): `beam_output.ex2` and `circle_output.ex2`.

- Each part is a separate database with its own active source: **Open** it, **Add** $\rightarrow$ **Mesh**, Displace with `dX` (every file has the same variable names, so one expression serves all), **Draw**; repeat. Colour the parts apart in the Mesh attributes (**Mesh color: Custom**) or colour each by its own field.

SEVERAL PARTS: OPERATORS, TIME, QUERIES

- “Apply operators to all plots” would put the operator on all mesh plots in one go, but also on the fluid (slide 22); untick and apply per plot, or tick, apply, and remove it from the fluid plot (chapter 21’s note).
- The time slider: the files written in the same run at the same interval have the same number of states and step together; **Controls** $\rightarrow$ **Database correlations** lines up files with different cadences by time (chapter 26).
- Querying two parts against each other (the gap between the beam’s top and the circle’s bottom in IBFE/08) is not a GUI query: two picks per frame, or the two-file script of chapter 32.
- Upstream ex7’s files also carry node sets (`Nodesets` in the variable list): the boundary ids that libMesh’s `build_square` gives the edges of each wall mesh. Its tether selects by the reference x coordinate (chapter 23), not by set; a Subset of the node sets shows the mesh edges that carry an id.
- One part per file is IBAMR’s choice, not Exodus’s: a single Exodus file can hold several element blocks (slide 38), which is what you get when one mesh file contains two bodies (chapter 21’s way C).

ELEMENT BLOCKS AND SETS: THE SUBSET PLOT



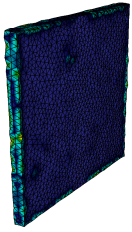
IBFE/05's gmsh triangle (the `triangle_gmsh.msh` run): the Subset plot of `ElementBlock` with the Mesh plot on top; one block, `triangle_1`: the physical surface "triangle" of `triangle.geo`, with the block id 1 appended by VisIt.

- **Add** → **Subset** → **ElementBlock** colours the elements by block; the Subset window lists the blocks and can hide them. The gmsh triangle of IBFE/05 has one block named `triangle`: libMesh copies a gmsh physical name into the subdomain name and the Exodus block name (VisIt shows it as `triangle_1`, name plus id); the course's shipped `triangle.msh` carries no physical name and its block is unnamed, shown by its id, like the plate's block `0`; the 3D plate's is `plate2_1`.

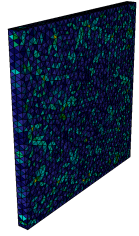
ELEMENT BLOCKS AND SETS (2): WHERE THEY COME FROM

- Blocks come from the mesh file: gmsh physical surfaces or volumes (chapter 18; libMesh maps each to a subdomain id and Exodus block), Exodus/Cubit blocks, or `elem->subdomain_id()` set in the program. The IBFE/10 two-material plate of chapter 22 chose its material by position instead, so its file has one block.
- Node sets and side sets (gmsh physical lines, Cubit nodesets/sidesets) appear as the scalar `Nodesets` and as Subset categories when present: upstream ex7's walls carry them, the plate does not.
- A Threshold on `ElementBlock` (a zonal integer) is the other way to keep one block.

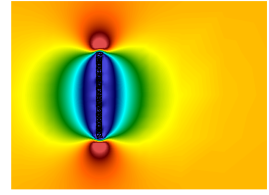
THREE DIMENSIONS: THE PLATE OF IBFE/10



`magnitude({dX_0, dX_1, dX_2})` on the displaced plate at $t = 0.5$: maximum 0.052 at the free edge; 3386 nodes, $11\,481$ tetrahedra, block `plate2_1`



`mesh_quality/aspect_gamma` of its tetrahedra at the reference state (1 to 4 shown; the range is 1.005 to 225: the slivers of chapter 22's note; 337 of the $11\,481$ tetrahedra are above 4)



the fluid slice with the plate's section (chapter 22)

- The 3D file (`viz_IB3d/output.ex2`) lists `X_0..X_2`, `dX_0..dX_2`, `U_0..U_2`, `F_0..F_2`, `coordx..coordz`; VisIt 3.5 does *not* build `dX`, `U`, `F`, `X` for this file (its expression list has only `mesh_quality` and `time_derivative` entries), while it does for the 2D files: in 3D the vector expression `{dX_0, dX_1, dX_2}` is yours to define, exactly as in 2013.

THREE DIMENSIONS: INSIDE THE MESH

- The Mesh plot of a volume mesh shows its outer surface; to see inside, **Operators** → **Selection** → **Threshold** or **Clip**, or a Slice through the mesh (which cuts the displaced mesh when the Displace operator comes first in the operator list: order matters, top of the list is applied first).
- The Displace operator accepts a 2-component `dX` in 3D without a word and displaces in x and y only (checked: the same x and y extents as with three components, z unchanged): the picture looks almost right and the error is easy to miss, so define `{dX_0, dX_1, dX_2}`.
- A Pseudocolor of a nodal field on a volume mesh colours the surface by the surface nodes' values; a Threshold on `mesh_quality/aspect_gamma` > 4 keeps the 337 slivers alone and shows where the mesher put them (the plate's edges and corners).
- Chapter 22 read the two-material plate's two displacements from this file; chapter 27 has the cameras, slices and isosurfaces that make such pictures, chapter 18 the element-size rules and chapter 22's sliver slide the shape rules the `mesh_quality` expressions check.

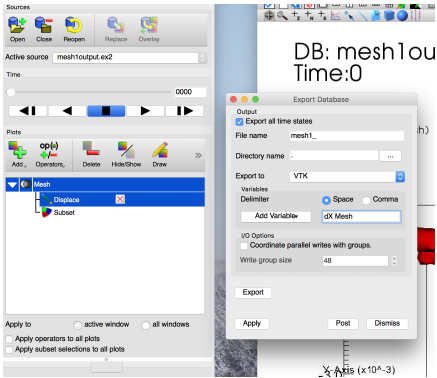
EXPORT: THE MESH WITH ITS FIELDS AS VTK

```
# File > Export database, or in the CLI:
e = ExportDBAttributes()
e.db_type = "VTK"; e.db_type_fullname = "VTK_1.0"
e.dirname = "/root/work/ch28_export"
e.filename = "plate"; e.variables = ("dX_0",)   # the call that
  wrote the file below
e.allTimes = 1; ExportDatabase(e)               # every state ...
   each overwriting the last (see right)
# one state: e.allTimes = 0; SetTimeSliderState(10); e.filename =
   "plate_0010"; ExportDatabase(e)
```

```
# what VisIt 3.5.0 wrote (headless): the file name is dropped,
  the file is <current directory>/.vtk
# vtk DataFile Version 4.2            # legacy ASCII, one time
  state
DATASET UNSTRUCTURED_GRID
FIELD FieldData 4                     # avtOriginalBounds,
  MeshName, CYCLE 0, TIME 4 (the last state)
POINTS 101 double                     # the reference positions (
  no Displace on the plot)
CELLS 130 520                         # the triangles
POINT_DATA 101
FIELD FieldData 1
dX_0 1 101 double                     # one array per exported
  variable (one asked for here)
```

- With the Mesh plot selected (“highlight the mesh”), **File** → **Export Database**: file name, directory, **Export to: VTK**, **Export all time states**, **Add variable** for each field to carry (the mesh goes anyway), **Apply**, then **Export**. The export writes the mesh *as plotted*: with the Displace operator on the active plot, the VTK points are the current positions; without it, the reference ones plus `dX`.
- Observed in 3.5.0 run headless: the `filename` and the directory are ignored and every call writes `.vtk` into the *current directory* of the VisIt process (chapter 33 found this; chapter 28’s test ran inside the export folder, where the two coincide), so **Export all time states** overwrites itself; export one state at a time and rename the file (`os.rename`) after each call. The export options (`GetExportOptions('VTK')`) choose legacy or XML, ASCII or binary.

EXPORT: WHEN AND WHAT FOR



Window: “3B – Analyzing your IBFE simulation” (2018), VisIt 2.12.2.

- The 2018 reason: KillDevil’s VisIt 2.12.3 had no Exodus reader, so the `.ex2` files were exported on a laptop and the VTK files read on the cluster (slide 45); “if you are doing the standard fiber-based IBAMR, you do not need to worry about this step” (a Silo run never needed it). VisIt 3.5 reads Exodus everywhere; the export is still the shortest way to an `.ex2` mesh in Python or MATLAB without netCDF (chapter 33: `pyvista` or the `vtk` module read the exported file in one line; `meshio` 5.3 cannot, because of the `FIELD` block VisIt writes).
- The 3.5 window is titled **Export Database**; its button is **Add variable**. Other export formats in the menu: Silo, Tecplot, Xmdv (columns of numbers), PLY and STL for the surface of a 3D mesh (a 3D-printable file of a deformed body, with the Displace operator on); the Silo points of a point-method run export the same way (a VTK polydata of points).
- The export takes the active plot’s data after its operators: a Box or Threshold before the export writes the selected part only, and a Slice of a 3D mesh writes the cut surface.

THE .curve FILE: A TIME QUERY OUT, INTO MATLAB OR PYTHON

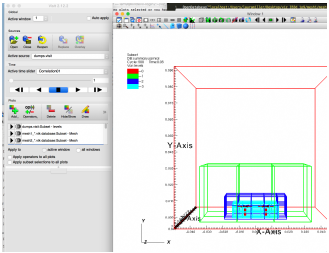
```
# window 2 active; File > Set save options: file type Curve; Save
# (CLI: SaveWindowAttributes().format = s.CURVE)
# Max_dX_mag
0 0
0.1000000000000001 0.05183466718831264
0.2000000000000001 0.1011884718693308
0.3000000000000002 0.1510825088362431
...
# Average_Value_dX_mag
0 0
...
```

```
% MATLAB (2018 deck, slide 16; a block per curve in the file)
C = importdata('plate_dX_queries.curve');
t = C.data(:,1);  y = C.data(:,2);  plot(t, y)
```

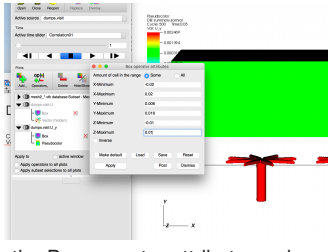
```
# Python: one block per curve; '#' lines are the headers
import numpy as np
blocks, cur = {}, None
for line in open("plate_dX_queries.curve"):
    if line.startswith("#"): cur = line[1:].strip(); blocks[cur]
  = []
    else: blocks[cur].append([float(v) for v in line.split()])
t, ymax = np.array(blocks["Max_dX_mag"]).T        # the other
  block: "Average_Value_dX_mag"
```

- A `.curve` file is text: a `#` header line with the curve's name, then `time value` pairs, one block per curve in the window. The 2018 deck's `importdata` reads the first block; with two curves in one window, split on the headers as the Python lines do, or save each curve from its own window.
- The same file comes from a **Do time curve** pick (slide 28) and from a Lineout (chapter 29).
- **Set save options** also offers **ULTRA** (another text curve format) and **VTK**; keep `.curve` for anything a script will read.
- Chapter 33 collects every way out: this file, the VTK export, the `hier_data` text, netCDF for `.ex2` directly, HDF5 for the fluid dumps.

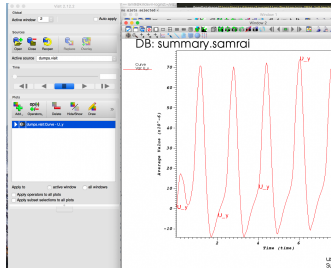
THE 2018 RECIPE: ANALYSING AN IBFE RUN AWAY FROM THE GUI



two exported parts (`mesh1_*.vtk`, `mesh2_*.vtk`) as Subset plots with the fluid



the Box operator attributes: a box above the structures



the averaged `U_y` against time in window 2

1. Open the `.ex2` files on the laptop, plot the mesh, displace it, **File** → **Export database**, all time states, VTK, variables `dX` and Mesh (slide 42).
2. Close VisIt, reopen, start recording (chapter 32), open `dumps.visit` and the VTK files (`mesh1_*.vtk database`), untick “apply to all”.

THE 2018 RECIPE (2): THE QUERY, THE FILE, THE CLUSTER

3. Pseudocolor of `U_y`; **Operators** → **Selection** → **Box** around the region of interest (“of whatever size you like”, all three ranges, kept the same from run to run: “what you pick does not matter too much as long as it’s consistent”); **Controls** → **Query over time options**, x-axis time; **Query** → **Average Value**, **Do Time Query**.
4. Window 2 active; **File** → **Set save options**, file type `curve`, **Save**; stop recording; search the Python for every path and change it to the cluster’s; run `visit -cli -nowin -s script.py` there.

- The run in the deck was a 3D two-coral case (`viz_IB3d`); the pictures on the previous slide are its screenshots, kept as the record of the recipe.
- Steps 2 and 4’s recording is chapter 32; step 3’s Eulerian average is chapter 29; chapter 31 has the cluster side (modules, batch VisIt). The deck’s own reading of the `.curve` file (“time is the first column, velocity is the second”) is right; the x-axis option is `DTime` in 3.x.
- What has changed: the cluster’s VisIt reads Exodus now, so step 1 is optional; the paths are still the thing to search for (“I recommend doing a search to find all paths”); the `.curve` import into MATLAB is unchanged (slide 44).

THE 2018 RECIPE (3): THE RECORDED SCRIPT

```
# the deck's slide 15, as recorded on the Mac (VisIt 2.12.2): three databases, two exported parts as Subset plots
OpenDatabase("localhost:/Users/lauramiller/Desktop/viz_IB3d_1p9/mesh1/mesh1_*.vtk database", 0)
OpenDatabase("localhost:/Users/lauramiller/Desktop/viz_IB3d_1p9/mesh2/mesh2_*.vtk database", 0)
OpenDatabase("localhost:/Users/lauramiller/Desktop/viz_IB3d_1p9/dumps.visit", 0)
ActivateDatabase("localhost:/Users/lauramiller/Desktop/viz_IB3d_1p9/mesh1/mesh1_*.vtk database")
AddPlot("Subset", "Mesh", 1, 0)
ActivateDatabase("localhost:/Users/lauramiller/Desktop/viz_IB3d_1p9/mesh2/mesh2_*.vtk database")
AddPlot("Subset", "Mesh", 1, 0)
DrawPlots()
SubsetAtts = SubsetAttributes()
SubsetAtts.colorType = SubsetAtts.ColorByMultipleColors  # ColorBySingleColor, ColorByMultipleColors, ColorByColorTable
SubsetAtts.colorTableName = "Default"
SubsetAtts.invertColorTable = 0
SubsetAtts.filledFlag = 1
SubsetAtts.legendFlag = 0
SubsetAtts.lineStyle = SubsetAtts.SOLID  # SOLID, DASH, DOT, DOTDASH
```

- “In this code, there are paths throughout. Be sure to search and replace all” (the Mac paths become the cluster’s); “stop recording, and cut and paste python commands to file”. A `*.vtk database` name opens a numbered series of VTK files as one time series. The recording itself, and the loop over states that saves a picture per frame (the deck’s slide 7), are chapter 32.

SCRIPTED: WHAT MADE THESE PICTURES

```
viz = "/root/work/0.19.0/ibfe01_HALF/viz_IB2d"
OpenDatabase(viz + "/dumps.visit"); OpenDatabase(viz + "/output.ex2")
DefineScalarExpression("dX_mag", "magnitude(dX)")   # dX exists already
ActivateDatabase(viz + "/dumps.visit"); AddPlot("Pseudocolor", "Omega")
ActivateDatabase(viz + "/output.ex2")
AddPlot("Pseudocolor", "dX_mag")
AddOperator("Displace"); d = DisplaceAttributes(); d.variable = "dX"; d.
  factor = 1.0
SetOperatorOptions(d)                                  # on the active
  plot only
AddPlot("Mesh", "Mesh"); AddOperator("Displace"); SetOperatorOptions(d)
DrawPlots(); SetTimeSliderState(10)
PickByNode(0, 0, ("X_0", "X_1", "dX_1", "U_1", "F_1")); print(
  GetPickOutput())   # (node, domain): keyword form PickByNode(domain=0,
  element=k, vars=(...)) is safer
SetActivePlots(1); print(Query("Max"))                 # the Pseudocolor
  plot
qt = GetQueryOverTimeAttributes(); qt.timeType = qt.DTime;
  SetQueryOverTimeAttributes(qt)
QueryOverTime("Max"); QueryOverTime("Average Value")  # -> window 2
AddOperator("Box"); b = BoxAttributes(); b.minx, b.maxx = 0.0, 1.0;
  SetOperatorOptions(b)
```

- Every GUI step on these slides has a one-line CLI form, and the GUI's **Controls** → **Command** window records them as you click (chapter 32, with the 2018 recording recipe).
- `AddOperator` and `SetOperatorOptions` act on the *active* plot (`SetActivePlots`); passing `1` as a second argument applies to all plots, the scripted form of the checkbox.
- Queries need the right active plot: `Query("Max")` on a Mesh plot gives "Cannot perform a Max query on variable Mesh".
- The whole script of this chapter (`wsl/final/visit_ch28.sh`) also exports the VTK and curve files and renders the 3D and Silo pictures; it runs headless (`visit -nowin -cli -s`).

THEN AND NOW

the decks said (2013–2020)	VisIt 3.5 with IBAMR 0.19
"open `dump.visit`"	the file is `dumps.visit`
create the vector expression `dX = {dX_0, dX_1, 0}`, save it as XML	VisIt builds `dX`, `U`, `F`, `X` from the components on opening; 2D files have two components; expressions you add are saved with the session
Operators → **Transforms** → **Project** to flatten the 2D mesh	not needed: the mesh is two-dimensional
"`vist_dump.xxxxx`: stores all the Cartesian grid data from each core"	`visit_dump.NNNNN`, listed by `dumps.visit`
x-axis "Time" in the query-over-time options	**Time** in the window, `DTime` in the attributes (**Cycle**, **Timestep** the others)
"**Control** → **Annotations**"	**Controls** → **Annotation**
"velocity exerted on the mesh"	the mesh's velocity `U`, interpolated from the fluid
"`sqrt(dX_0*dX_0 + dX_1*dX_1)` gives the radial component"	the in-plane displacement magnitude; the radial part is a projection (slide 35)
the cluster's VisIt cannot read `.ex2`: export as VTK first	the Exodus reader ships in every standard VisIt binary (a cluster's own build may omit it: check **File** → **File information** there); export when a script wants VTK
"ex2: Cavity flow with mesh at bottom / ex3: Cavity flow with mesh at bottom / ex6: Flow past beam and block"	chapter 25's table: ex2 is a soft plate pushed by a traction difference, ex3 a soft block under a sliding lid, ex6 the Turek–Hron beam (the shell test is ex1); slide 50
Python 2 CLI, `module add visit/2.12.3`	Python 3 inside VisIt 3.x; chapter 31 for the cluster modules

THE 2013 EXAMPLES LIST, AND WHAT THE EXAMPLES ARE

the 2013 document's "Examples List"	chapter 25's description of the same upstream IBFE example
"ex0: Rubber band"	an elastic ring at rest in a periodic box with the exact pressure jump set: a static equilibrium test
"ex1: Stretched rubber band"	the same ring started as an ellipse: the dynamic shell test of Griffith and Luo
"ex2: Cavity flow with mesh at bottom"	a soft plate across a 2×1 box pushed by a traction difference, no tether
"ex3: Cavity flow with mesh at bottom"	a soft block on the floor of a lid-driven cavity: a solid deformed by a shear flow
"ex4: Cavity flow with deformable ball"	a soft disc in a lid-driven cavity (Griffith and Luo's Fig. 10)
"ex5: Flow past cylinder"	a cylinder at $\mathrm{Re} = 200$: vortex shedding, C_D and C_L; aborts as shipped (chapter 25)
"ex6: Flow past beam and block (square with long rectangular tail)"	a rigid disc with an elastic beam behind it in a channel: the Turek–Hron FSI3 benchmark

- The 2013 document's note "Can have multiple files of this type for different parts of object (see ex6)" is right: ex6 writes one Exodus file per part (slide 36). Its two identical lines for ex2 and ex3 are the copy error chapter 25 corrected; the examples after ex6 (ex7–ex11) came later.

WHAT CAN GO WRONG (1)

symptom	cause and fix
"An invalid variable (dX) was specified" and an empty window	**Apply operators to all plots** was ticked: the fluid plot got the Displace operator; remove it from that plot and untick
the mesh sits where it started while the fluid moves	no Displace operator on that plot, or its variable is **default**; every plot that should move needs its own
the mesh is drawn as a copy shifted by its reference coordinates	the Displace variable is `X` (the position) instead of `dX`: VisIt draws $\mathbf{s} + \mathbf{X}$ instead of $\mathbf{s} + \mathbf{dX}$
in 3D the mesh moves in x and y but not in z	a 2-component `dX`, accepted silently: use `{dX_0, dX_1, dX_2}`, which 3.5 does not define for you in 3D
"Cannot perform a Max query on variable Mesh"	queries need the Pseudocolor (or Vector) plot active, not the Mesh plot
"An invalid variable (`<name>_mesh`) was specified"	the structure has no springs, so no `_mesh` was registered: draw `<name>_vertices`
no `output.ex2` in `viz_IB2d`	`viz_dump_interval = 0` (no dumps at all), or the program never reached its first dump; the `"ExodusII"` word in `viz_writer` is not the switch (slide 15)
no `lag_data.visit`	`viz_writer` lacks `"Silo"` (and an IB/01-style program crashes at the first dump)

WHAT CAN GO WRONG (2)

symptom	cause and fix
`lag_data.visit` has only positions	by design: forces and velocities of a point-method run are in `hier_data` (chapter 11); the displacement is a cross-time expression (slide 12)
new states do not appear while a run is going	VisIt read the file once: **File** → **ReOpen** (the Exodus file grows in place; the `.visit` lists grow by lines)
after a restart the plate file starts over at state 1	the program did not call `append(from_restart)`; the old states are gone (rerun from the restart with it, or keep the first file's copy)
the fluid and the mesh are out of step	different dump intervals or a declined correlation: **Controls** → **Database correlations**
a Subset or Threshold leaves nothing	**Threshold**'s **Show zone if: Part in range** / **All in range** (`zonePortions`) or an empty box; check the ranges against **SpatialExtents**
a query returns nothing	the plot is hidden (a hidden plot cannot be queried), or the Mesh plot is the active one
the Box keeps the wrong part	operator order: Displace before Box selects by current position, after it by reference position
a Pseudocolor of the mesh looks blocky in 3D	only the outer faces are drawn; Clip or Threshold to look inside
the exported VTK file is called `.vtk`, in the wrong folder	3.5.0 headless drops the file name and the directory (it writes into VisIt's current directory); rename after each export (slide 42; chapter 33's `resample_export.py` does it)
`mesh_quality/aspect` is zero everywhere	it is not defined for triangles in 3.5; use `minimum_angle`, `condition` or `shape` in 2D, `aspect_gamma`, `condition` or `shape` in 3D

EXERCISES

1. Run IBFE/01 with `MOTION = "ROTATE"` and draw the plate at $t = 1$, 2 and 4 over the vorticity, coloured by `magnitude(U)`; which end moves fastest after the release?
2. Pick the two tips (the nodes at $x = \pm 0.5$ on the centre line) with **Do time curve** and reproduce the README's HALF numbers ($y = +0.5$ at $t = 1$, -0.5 at $t = 3$ for the tethered tip; the lag of the free one).
3. Query the average of `magnitude(F)` over the free half (Box, $x < 0$) and over the tethered half against time; where does the tether work hardest?
4. On IBFE/08 (two files): draw the beam coloured by `dX_1` and the circle by `magnitude(U)`, each displaced; then colour both by `ElementBlock` and explain the legend.
5. On IB/13: build the cross-time displacement of slide 12 and query its maximum over the cycle; compare with the drift chapter 9a's figures measured from the `X.*` files.
6. Export IBFE/01's plate at $t = 1$ to VTK with `dX` and `F`, and read the file back in Python (chapter 33's reader, or `meshio`); check the tip's `dX_1` against the Pick output.
7. Threshold the 3D plate of IBFE/10 on `mesh_quality/aspect_gamma` above its 99th percentile and count the elements; where are they (chapter 22's sliver note)?
8. Replace step 5 of slide 19 by the Project operator on purpose: does anything change? Then set the Displace multiplier to 10 and say what the picture now shows.

RULES OF THUMB

- Points: `lag_data.visit` has positions only; the numbers live in `hier_data`. Elements: `output.ex2` has everything, one file per part.
- Untick **Apply operators to all plots** before anything else; then Displace with `dX`, one operator per plot that moves.
- Colour the structure by its own fields (`dX`, `U`, `F` and expressions); draw the Mesh plot on top with opaque mode off.
- Pick for one node, query for a field, select before you query a part; put the selection on the plot you query.
- `mesh_quality` at the reference state judges the mesher, at a later state the deformation.
- Export VTK for scripts, `.curve` for time series; save the session next to the figure.

SUMMARY

1. A point-method run writes its structures to Silo (`lag_data.visit`: `<name>_vertices` as points, `<name>_mesh` as spring edges, one file per rank but every structure in rank 0's, in Lagrangian order; no fields), an IBFE run to ExodusII (`output.ex2` per part: the reference mesh and the nodal `X`, `dX`, `U`, `F` at every dump).
2. The finite-element mesh is drawn where it is by the Displace operator with `dX`, which VisIt 3.x already defines; the operator must sit on each plot that should move and must not reach the fluid's.
3. The mesh's fields colour it (Pseudocolor), move with it (Vector) and judge it (`mesh_quality`); expressions combine them; selections cut it down; Pick and the time queries turn it into numbers and curves.
4. IB points have no `dX`, but a cross-time expression builds the displacement from the positions; the domain Subset has one domain per structure, not per rank.
5. Exported VTK files and `.curve` text are the hand-over to Python and MATLAB (chapter 33) and the scripted versions of all of this are chapter 32.

Next: chapter 33, converting between data types and exporting to MATLAB and Python; then the IB2d chapters, then chapter 27 (3D), 29 (data analysis), 30 (movies), 31 (the cluster) and 32 (the command line).

RECORDINGS AND SCREENSHOTS

- Shannon Jones's YouTube video on visualizing the finite-element mesh (`watch?v=Kj_MACemSd0`), referenced by every re-issue of the 2013 recipe from 2018 on (the 2017 IBFE Read Me: "The youtube tutorial by shannon has a mistake, but this is corrected in the google slides"); and her channel's six VisIt how-tos (the 2017 "Visit Analysis and Python" Read Me; chapter 32). Listed, not incorporated.
- The 2020 Zoom recording of IBFE Lecture 1 (*BeamExample*, 20 May 2020), whose slides 18–26 are the 2020 form of this recipe, is in the front matter's list (chapter 0.1).
- The window screenshots here are the 2013, 2014 and 2018 decks' (VisIt 2.x, 2.7.2 and 2.12.2 on a Mac), credited on each slide; the re-shoot list in `plan/visit-screenshots-todo.md` now has a chapter 28 section (the Displace window with its Vectors submenu, the Pick window, the Query window with "All queries-over-time" and "Do Time Query", the Export database window, all in 3.5).

REFERENCES AND CREDITS

- A. Hoover, *IBAMR Finite Element Visualization with VisIt* (Miller Lab, 23 May 2013) and *5-IBFE Tutorial: Visualizing Lagrangian Data* (2014); B. Griffith's description of the Exodus output (IBAMR users' list, quoted in the April 2013 MATH 769 deck); S. Jones's video how-tos; the 2018 MATH 769 deck *3B – Analyzing your IBFE simulation*.
- IBAMR 0.19.0 source: `ibtk/src/lagrangian/LSiloDataWriter.cpp` and `include/ibtk/LSiloDataWriter.h` (file names, `registerMarkerCloud`, `registerUnstructuredMesh`, `registerVariableData`); `src/IB/IBRedundantInitializer.cpp` (`initializeLSiloDataWriter`: the `_vertices`, `_mesh`, `_xmesh`, `_rod_mesh` names); `src/IB/FEMechanicsBase.cpp` (the systems and their variables); `examples/IBFE/explicit/ex0/example.cpp` (`ExodusII_IO`, `append`, `write_timestep`). https://github.com/IBAMR/IBAMR.
- libMesh, `ExodusII_IO` (https://libmesh.github.io/doxygen/classlibMesh_1_1ExodusII__IO.html); Silo (https://silo.readthedocs.io/); VisIt 3.5.0 (https://visit-dav.github.io/visit-website/) and its user manual's chapters on the Displace operator, expressions (`conn_cmfe`, `pos_cmfe`, `coords`), Pick, Query over time, the Selection operators, Export database and the Curve format.
- Data: this series' IBFE/01 (FULL and HALF), IBFE/08, IBFE/10, IB/13 and IB/08 runs and upstream IBFE ex7, IBAMR 0.19.0, four cores; pictures by `wsl/final/visit_ch28.sh`; the exported VTK and curve files as shown.